

AKADEMIA NAUK STOSOWANYCH W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA ZAAWANSOWANE PROGRAMOWANIE

...Algorytm listy dwukierunkowej z zastosowaniem GitHub...

Autor:
Wiktoria Fałowska

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2024

Spis treści

1. Ogólne określenie wymagań	4
1.1. Wstęp	4
1.2. Wymagania	4
1.3. Cel projektu	5
2. Analiza problemu	6
2.1. Lista Dwukierunkowa- Czym jest?	6
2.2. Działanie listy dwukierunkowej	7
3. Projektowanie	8
3.1. Narzędzia	8
3.2. Diagram klas dla listy dwukierunkowej	9
4. Implementacja	10
4.1. Najważniejsze fragmenty kodu	10
4.1.1. Struktura węzła listy	10
4.1.2. Dodawanie elementów	11
4.1.3. Usuwanie elementu na wskazanym indeksie	12
4.1.4. Wyświetlanie listy	13
4.2. Narzędzie GIT i integracja	14
4.2.1. Cofnięcie się o kilka commitów.	14
4.2.2. Nowy branch	15
4.2.3. Usunięcie, revert, reset	17
4.2.4. Dodawanie zmian w osobnych folderach	19
4.2.5. Doxyfile	21
5. Wnioski	22
Literatura	23
Spis rysunków	24
Spis tabel	25

1. Ogólne określenie wymagań

1.1. Wstęp

Celem zadania jest napisanie programu w C++ implementującego listę dwukierunkową na sterzie, w formie klasy, wraz z zestawem funkcji umożliwiających operacje na tej liście. Program powinien być przetestowany w funkcji main, a projekt wymaga także dokumentacji i zarządzania wersjami kodu za pomocą GitHub.

1.2. Wymagania

Wymagania dotyczące funkcji listy dwukierunkowej:

- Dodawanie elementów
- Usuwanie elementów
- Wyświetlanie elementów
- Inne operacje

Wymagania techniczne:

- Dokumentacja:
Program musi mieć wygenerowaną automatycznie dokumentację przy użyciu narzędzia „doxygen” i zapisaną w formacie PDF za pomocą latex. Do projektu musi być również dołączona dokumentacja w latex, zgodnie z szablonem dostarczonym przez prowadzącego.
- Praca z GitHubem:
Do zarządzania wersjami kodu należy używać GitHuba. W trakcie realizacji projektu wymagane jest wykonanie co najmniej 5 commitów (np. cofnięcie, usunięcie lub dodanie).

Kolejnymi ważnym punktem jest pobranie projektu z GitHuba w innej lokalizacji na komputerze, wprowadzenie zmian i zapisanie tych zmian w repozytorium. Następnie pobranie tych zmian w oryginalnej lokalizacji.

1.3. Cel projektu

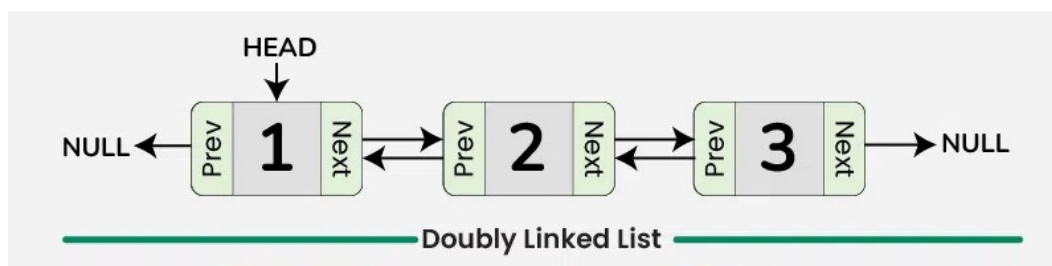
Celem projektu jest zdobycie praktycznych umiejętności w implementacji listy dwukierunkowej w C++ oraz pracy z systemem kontroli wersji Git. Projekt pozwala na naukę zarządzania zmianami w kodzie, cofania się do poprzednich wersji i synchronizacji projektu między różnymi lokalizacjami. Ważnym aspektem jest również tworzenie dokumentacji technicznej przy użyciu narzędzia doxygen. Projekt jest także idealnym przykładem do zapoznania się z tworzeniem repozytorium GitHub.

2. Analiza problemu

Lista dwustronnie powiązana jest bardziej złożoną strukturą danych niż lista jednostronnie powiązana, ale oferuje kilka zalet. Główną zaletą listy dwustronnie powiązanej jest to, że umożliwia ona wydajne przechodzenie listy w obu kierunkach. Dzieje się tak, ponieważ każdy węzeł na liście zawiera wskaźnik do poprzedniego węzła i wskaźnik do następnego węzła. Umożliwia to szybkie i łatwe wstawianie i usuwanie węzłów z listy, a także wydajne przechodzenie listy w obu kierunkach.

2.1. Lista Dwukierunkowa- Czym jest?

Lista dwustronnie powiązana to struktura danych składająca się z zestawu węzłów, z których każdy zawiera wartość i dwa wskaźniki, jeden wskazujący na poprzedni węzeł na liście i jeden wskazujący na następny węzeł na liście. Umożliwia to wydajne przechodzenie listy w obu kierunkach, dzięki czemu nadaje się do zastosowań, w których wymagane są częste wstawiania i usuwania.



Rys. 2.1. Reprezentacja listy dwukierunkowej.

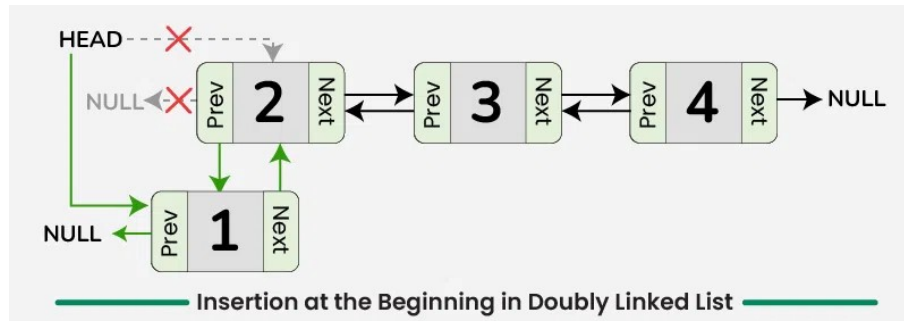
Na zdjęciu 2.1 (s. 6) ¹.przedstawiono listę dwukierunkową (doubly linked list), która składa się z trzech węzłów o wartościach 1, 2 i 3. Każdy węzeł jest połączony z sąsiadującymi węzłami zarówno do przodu, jak i do tyłu za pomocą wskaźników, co umożliwia poruszanie się w obu kierunkach w liście. Kluczowymi elementami listy są węzły, a każdy węzeł zawiera:

- Pole Prev – wskaźnik na poprzedni węzeł (lub NULL w przypadku pierwszego węzła, czyli głowy listy).
- Pole Dane – przechowuje wartość węzła, w tym przypadku odpowiednio 1, 2 i 3.
- Pole Next – wskaźnik na następny węzeł (lub NULL w przypadku ostatniego węzła).

¹Ilustracja pomocnicza-lista dwukierunkowa[1].

2.2. Działanie listy dwukierunkowej

Na ilustracji 2.2 (s. 7) ² przedstawiono operację wstawiania nowego elementu na początku listy dwukierunkowej.



Rys. 2.2. Wstawianie na początku listy dwukierunkowo powiązanej.

Działanie:

- Nowy element (1) jest wstawiany na początek listy. Jego pole Next wskazuje na dotychczasowy pierwszy element (2), a pole Prev ustawione jest na NULL, ponieważ staje się on nową głową listy.
- Dotychczasowy pierwszy element (2) aktualizuje swoje pole Prev, aby wskazywało na nowo wstawiony węzeł (1).
- Głowa listy zostaje przesunięta z węzła 2 na węzeł 1, co oznacza, że nowy węzeł staje się nową głową.
- Reszta listy (elementy 3 i 4) pozostaje niezmienną, z aktualnymi wskaźnikami na poprzedni i następny element.

W ten sposób nowy element zostaje dodany na początek listy, a wskaźniki w pozostałych węzłach są odpowiednio zaktualizowane, aby zachować integralność struktury listy dwukierunkowej.

²Ilustracja pomocnicza-lista dwukierunkowa-wstawianie[1].

3. Projektowanie

3.1. Narzędzia

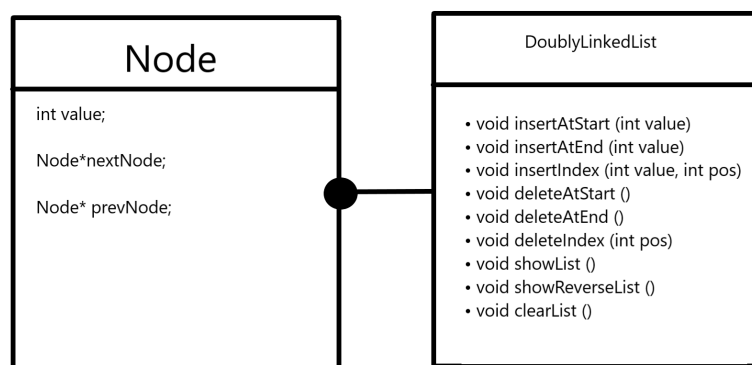
Nasz projekt posiadał kilka podstawowych oraz niezbędnych narzędzi potrzebnych do wykonania zadania.

- Jako główny język programowania użyto C++, który umożliwia implementację dynamicznych struktur danych, takich jak lista dwukierunkowa.
- Do kompilacji kodu oraz jego debugowania zastosowano środowisko programistyczne Visual Studio 2022, korzystając z kompilatora MSVC (Microsoft Visual C++).
- System kontroli wersji Git służył do zarządzania wersjami kodu i śledzenia zmian w projekcie, natomiast repozytorium kodu zostało umieszczone na platformie GitHub, która umożliwia łatwą współpracę, przechowywanie oraz synchronizację projektu.
- Do generowania dokumentacji technicznej projektu użyto narzędzia Doxygen, które automatycznie tworzy dokumentację na podstawie komentarzy w kodzie. Dokumentacja ta może być generowana w formacie HTML lub PDF.
- Zarządzanie repozytorium Git odbywało się przy użyciu terminala Git Bash oraz wbudowanego terminala w Visual Studio, co pozwalało na wygodne wykonywanie operacji Gita bez opuszczania środowiska programistycznego.

3.2. Diagram klas dla listy dwukierunkowej

Obrazek 3.1 (s. 9) przedstawia diagram ilustrujący strukturę listy podwójnie powiązanej, która składa się z dwóch głównych komponentów³:

- klasy Node (węzeł), która zawiera trzy elementy:
 - int value; — wartość całkowita reprezentująca dane przechowywane w węźle.
 - Node* nextNode; — wskaźnik na następny węzeł w liście.
 - Node* prevNode; — wskaźnik na poprzedni węzeł w liście.
- klasy DoublyLinkedList (lista podwójnie powiązana)-definiuje kilka metod do manipulacji listą:
 - void insertAtStart(int value); — wstawia nowy węzeł z podaną wartością na początku listy.
 - void insertAtEnd(int value); — wstawia nowy węzeł z podaną wartością na końcu listy.
 - void insertIndex(int value, int pos); — wstawia nowy węzeł z podaną wartością na określonym indeksie w liście.
 - void deleteAtStart(); — usuwa węzeł na początku listy.
 - void deleteAtEnd(); — usuwa węzeł na końcu listy.
 - void deleteIndex(int pos); — usuwa węzeł na określonym indeksie w liście.
 - void showList(); — wyświetla zawartość listy w kierunku od początku do końca.
 - void showReverseList(); — wyświetla zawartość listy w kierunku od końca do początku
 - void clearList(); — usuwa wszystkie węzły z listy.



Rys. 3.1. UML

³Źródło pomocnicze.[2].

4. Implementacja

W tym projekcie zaimplementowaliśmy listę dwukierunkową w języku C++. Kluczowym elementem jest stworzenie klasy Node, która reprezentuje pojedynczy węzeł listy, oraz klasy DoublyLinkedList, która zarządza całą listą i zawiera metody do manipulacji węzłami.

4.1. Najważniejsze fragmenty kodu

4.1.1. Struktura węzła listy

Węzeł listy widoczny 1 (s. 10) oznaczany jest przez klasę Node, która zawiera zmienną value przechowującą wartość oraz dwa wskaźniki: nextNode i prevNode na następny i poprzedni element.

```
1 class Node {
2 public:
3     int value;
4     Node* nextNode;
5     Node* prevNode;
6
7     Node(int value) {
8         this->value = value;
9         this->nextNode = nullptr;
10        this->prevNode = nullptr;
11    }
12};
```

Listing 1. Klasa Node

4.1.2. Dodawanie elementów

Aby dodać element na pierwszym miejscu (listing 2 (s. 11)), musimy stworzyć nowy węzeł, a następnie zaaktualizować węzły w taki sposób, aby struktura dalej była poprawna.

```
1 void insertAtStart(Node*& head, int value) {  
2     Node* newNode = new Node(value);  
3  
4     if (head == nullptr) {  
5         head = newNode;  
6         return;  
7     }  
8  
9     newNode->nextNode = head;  
10    head->prevNode = newNode;  
11    head = newNode;  
12 }  
13 };
```

Listing 2. Dodawanie elementów na początku listy

Aby dodać element na końcu listy (listing 3 (s. 11)) trzeba przejść przez całą listę, a następnie wstawić nowy węzeł na końcu.

```
1 void insertAtEnd(Node*& head, int value) {  
2     Node* newNode = new Node(value);  
3  
4     if (head == nullptr) {  
5         head = newNode;  
6         return;  
7     }  
8  
9     Node* temp = head;  
10    while (temp->nextNode != nullptr) {  
11        temp = temp->nextNode;  
12    }  
13  
14    temp->nextNode = newNode;  
15    newNode->prevNode = temp;  
16 }  
17 };
```

Listing 3. Dodawanie elementów na końcu listy

4.1.3. Usuwanie elementu na wskazanym indeksie

Aby usunąć element na danym indeksie trzeba do niego przejść od początku listy po czym usunąć go i zaaktualizować wskaźniki tak aby dalej lista działała (listing 4 (s. 12)).

```
1 void deleteIndex(Node*& head, int pos) {
2     if (head == nullptr) {
3         cout << "Lista jest pusta" << endl;
4         return;
5     }
6
7     if (pos == 0) {
8         deleteAtStart(head);
9         return;
10    }
11
12    Node* temp = head;
13    for (int i = 0; temp != nullptr && i < pos; i++) {
14        temp = temp->nextNode;
15    }
16
17    if (temp == nullptr) {
18        cout << "Zły indeks" << endl;
19        return;
20    }
21
22    if (temp->nextNode != nullptr) {
23        temp->nextNode->prevNode = temp->prevNode;
24    }
25    if (temp->prevNode != nullptr) {
26        temp->prevNode->nextNode = temp->nextNode;
27    }
28    delete temp;
29 }
30 };
```

Listing 4. Usuwanie elementu

4.1.4. Wyświetlanie listy

Metoda wyświetlająca całą listę przechodzi przez wszystkie Nody od początku do końca (listing 5 (s. 13)).

```
1 void showList(Node* head) {  
2     Node* temp = head;  
3     if (temp == nullptr) {  
4         cout << "Lista jest pusta" << endl;  
5         return;  
6     }  
7     cout << "Lista: ";  
8     while (temp != nullptr) {  
9         cout << temp->value << " ";  
10        temp = temp->nextNode;  
11    }  
12    cout << endl;  
13 }
```

Listing 5. lista

Oto przykładowy wynik poprawnie działającego programu (listing 6 (s. 13)):

```
1 Dodajmy element na poczatku:  
2 Lista: 1  
3 Dodajmy element na koncu:  
4 Lista: 1 3  
5 Dodajmy element na indeks 1:  
6 Lista: 1 2 3  
7 Wyświetlmy liste na odwrot:  
8 Lista odwrotnie: 3 2 1  
9 Dodajmy jeszcze kilka wartosci:  
10 Lista: 1 2 3 6 13 10 0  
11 Usunmy element na indeksie 4:  
12 Lista: 1 2 3 6 10 0  
13 Usunmy pierwszy element:  
14 Lista: 2 3 6 10 0  
15 Usunmy ostatni element:  
16 Lista: 2 3 6 10  
17 Czyscimy cala liste:  
18 Lista jest pusta
```

Listing 6. Wynik

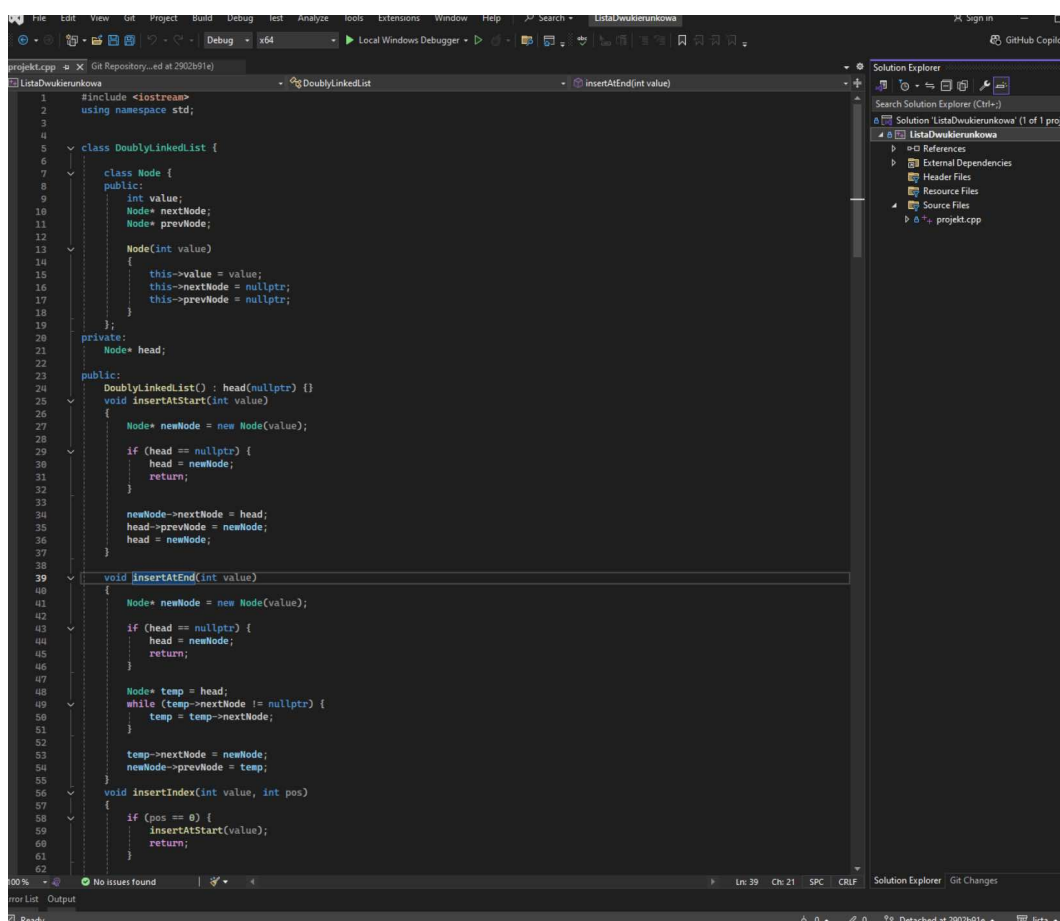
4.2. Narzędzie GIT i integracja

Po zainstalowaniu Git ⁴ możemy go używać w terminalu (Git Bash lub wbudowany w Visual Studio). Najbardziej przydatne oraz najważniejsze polecenia:

- git init – inicjalizuje nowe repozytorium Git.
- git add . – dodaje wszystkie zmienione pliki do obszaru stage.
- git commit -m "commit" – zapisuje zmiany w lokalnym repozytorium.
- git push – przesyła zmiany do zdalnego repozytorium na GitHubie.

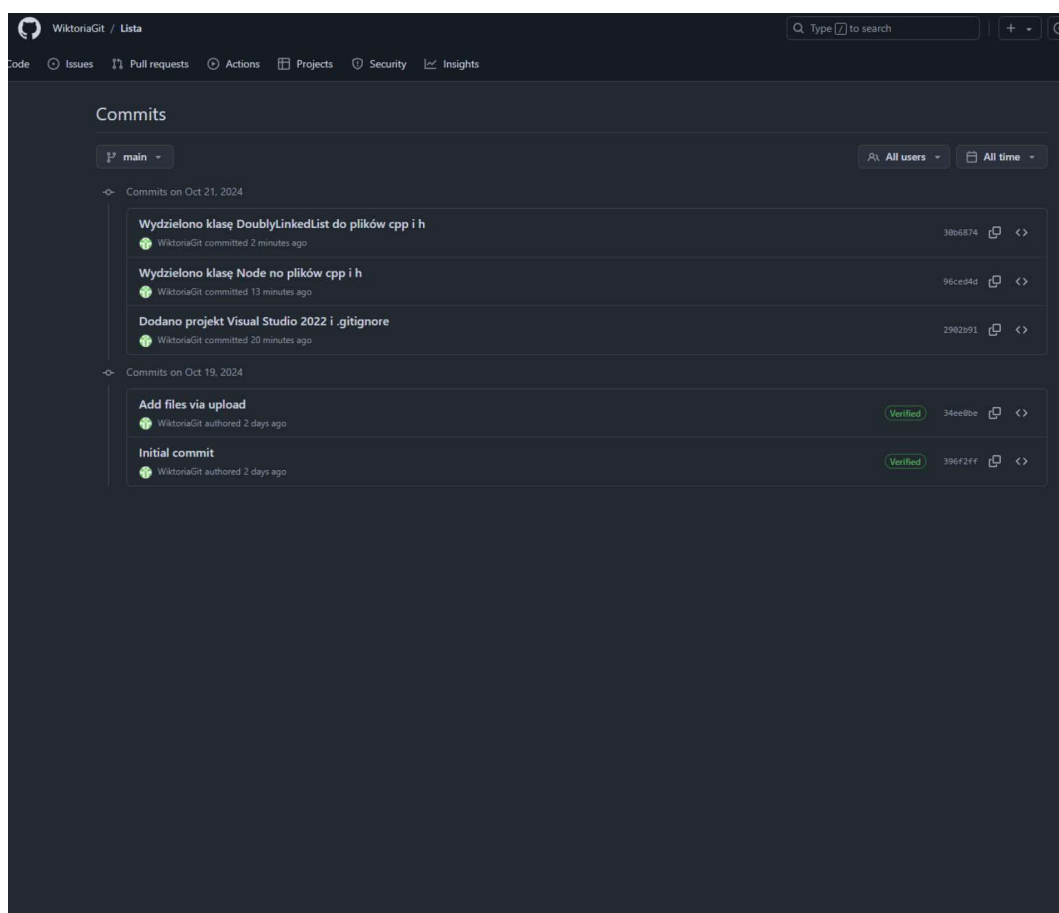
4.2.1. Cofnięcie się o kilka commitów.

Na dole po prawej stronie VS jest pokazane ID commita 4.1 (s. 14) , zgodny z tym co jest widoczne na GitHubie. 4.2 (s. 15)



Rys. 4.1. Cofnięcie 1.1

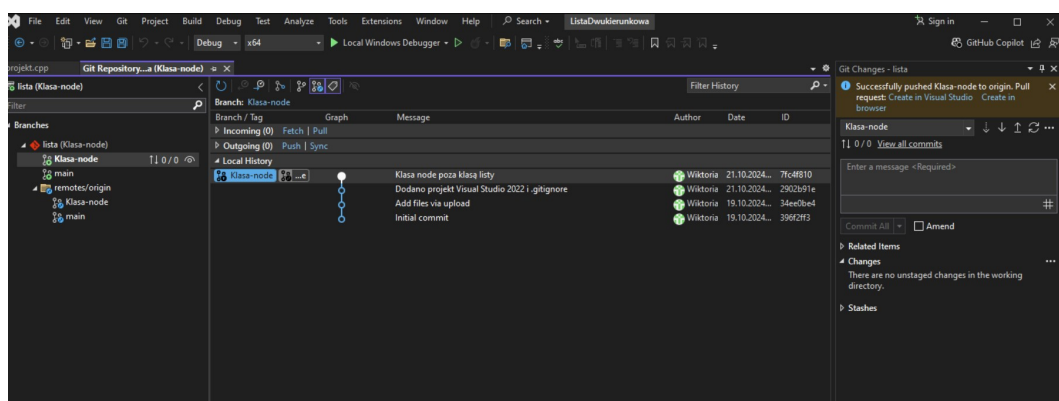
⁴Źródło informacji[3].



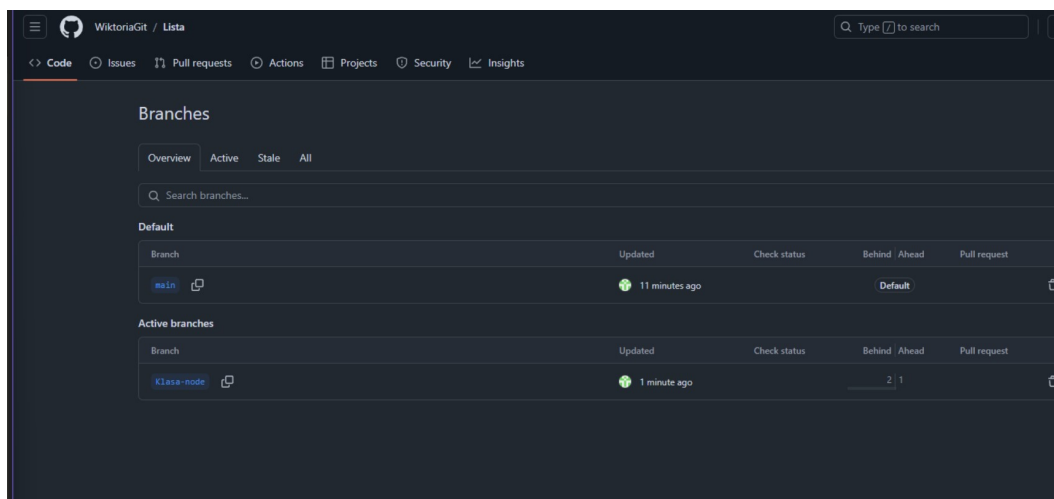
Rys. 4.2. Cofnięcie 1.2

4.2.2. Nowy branch

Nowy branch na który wprowadzamy osobne zmiany. 4.3 (s. 15) oraz 4.4 (s. 16)

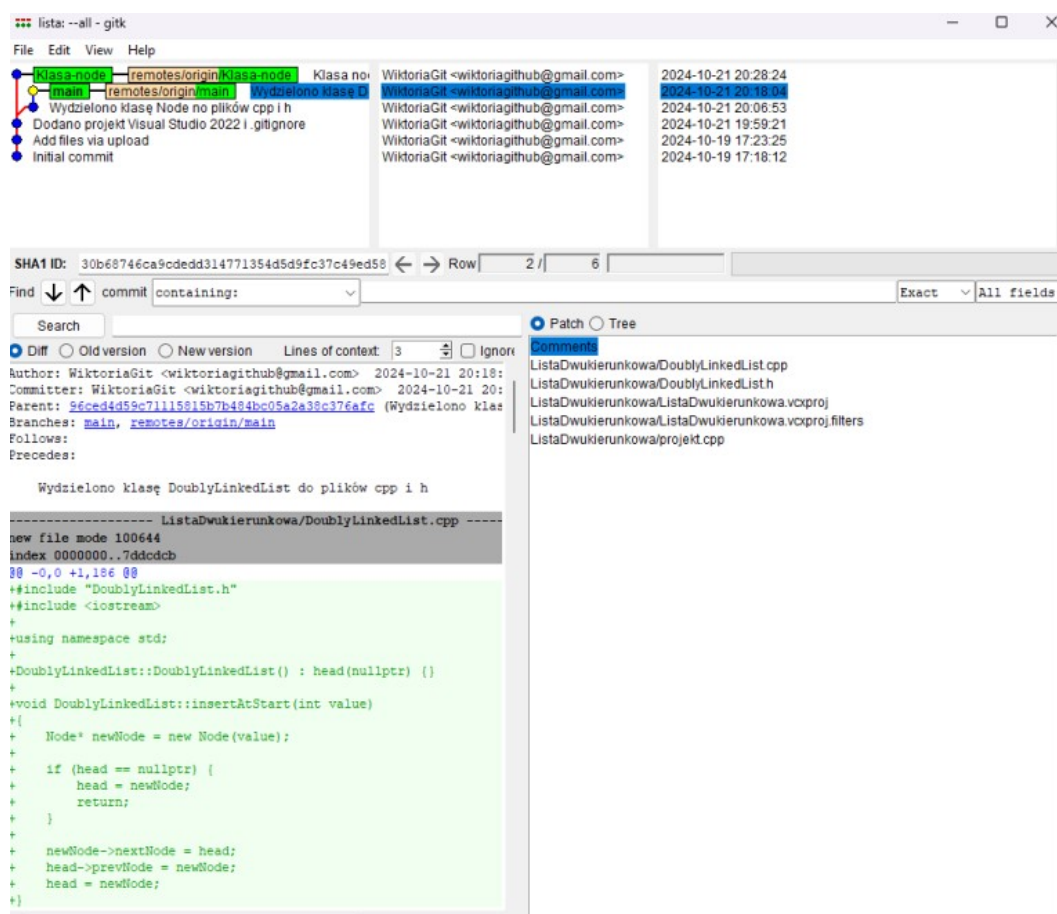


Rys. 4.3. Branch 1.1



Rys. 4.4. Branch 1.2

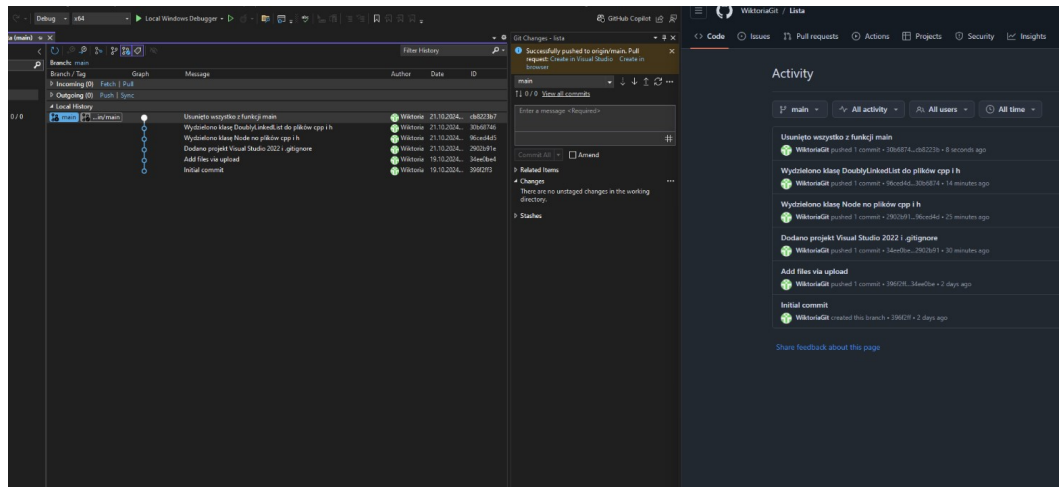
Wizualizacja branchy za pomocą programu Git Gui 4.5 (s. 16) (opcja Repository, Visualize All Branch History). Pokazuje branch-ke którą stworzyliśmy (idzie równoległe do Main).



Rys. 4.5. Wizualizacja branchy

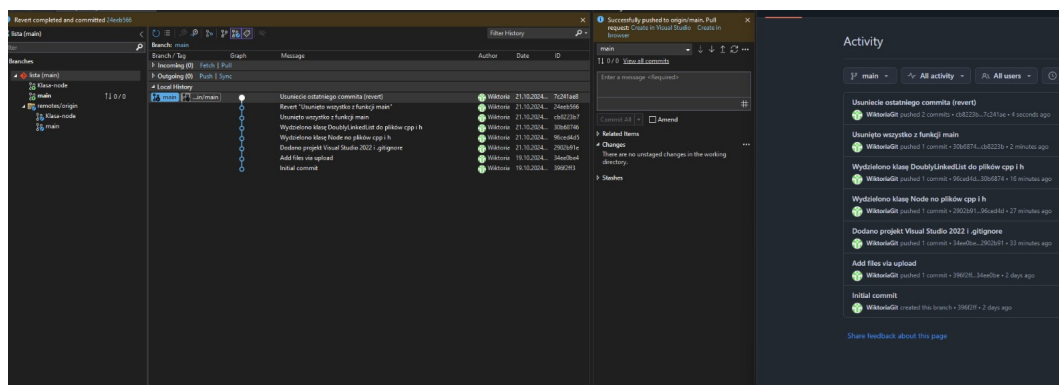
4.2.3. Usunięcie, revert, reset

Commit który będziemy usuwać/revertować. 4.6 (s. 17)



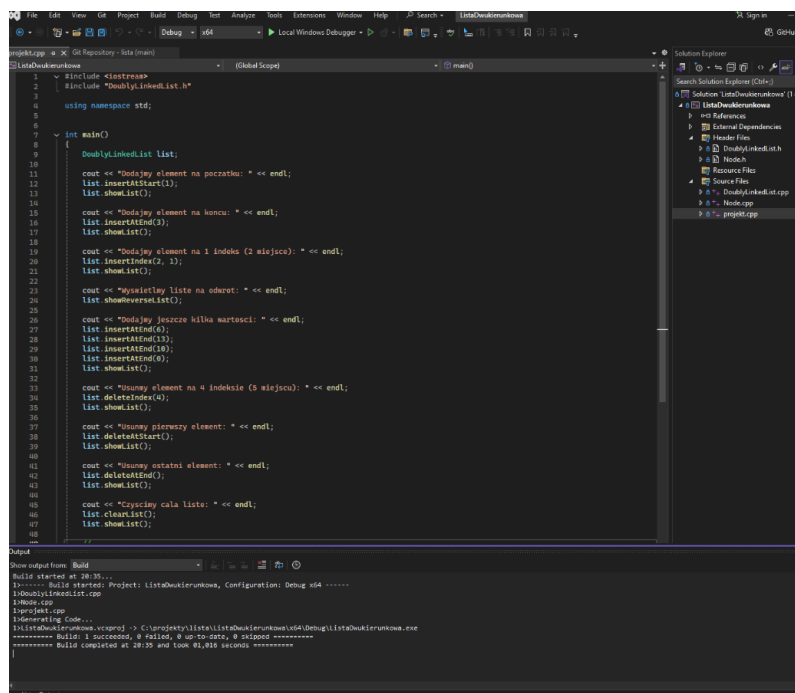
Rys. 4.6. Commit-usuwany

Revert poprzedniego commmita (zmiany wracają do poprzednich stanów).4.7 (s. 17)



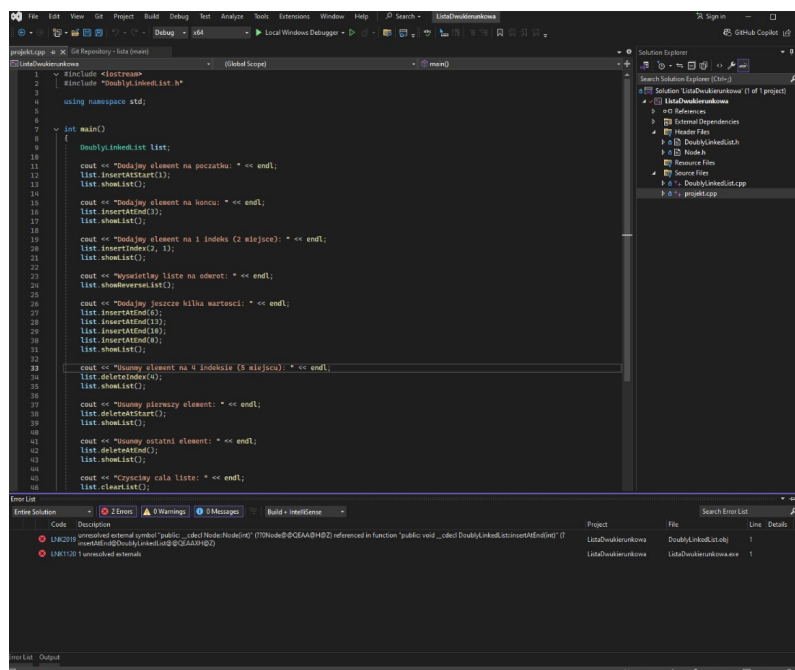
Rys. 4.7. Revert commita

Projekt się kompiluje poprawnie.4.8 (s. 18)



Rys. 4.8. Projekt

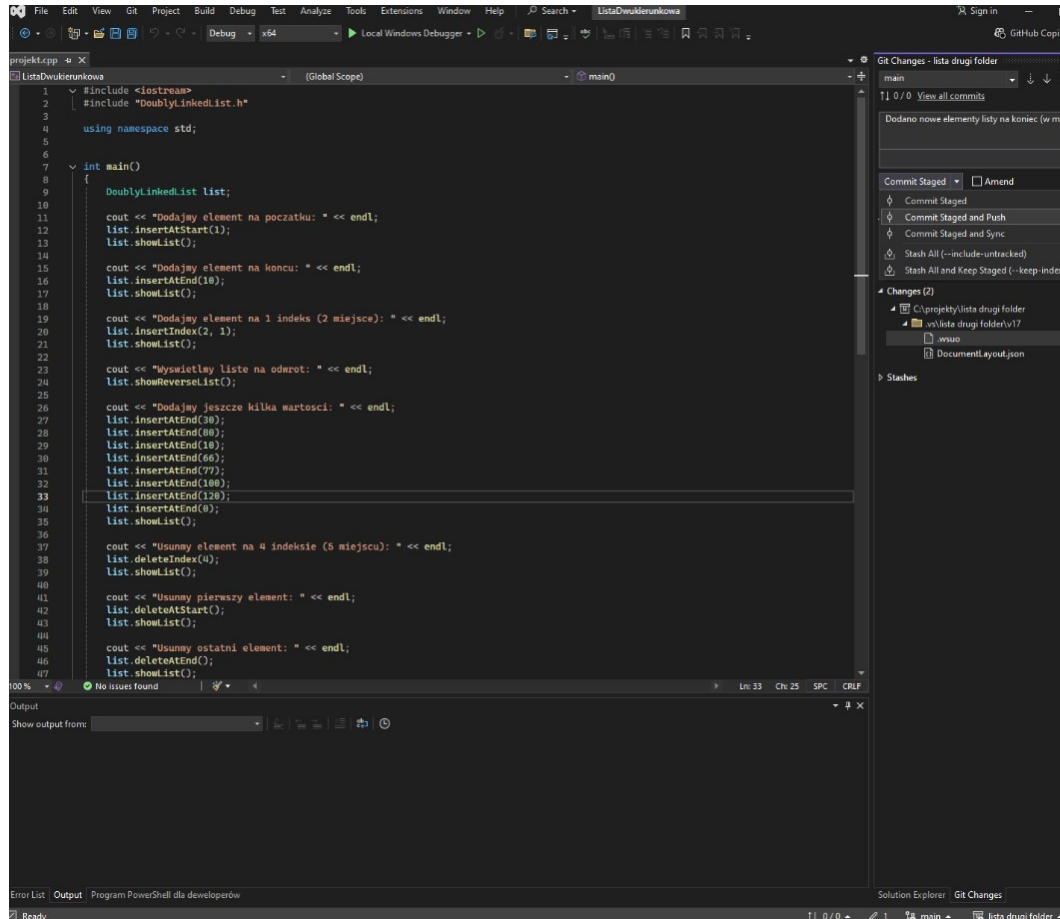
Usuneliśmy plik Node.cpp- projekt sie nie kompiluje.4.9 (s. 18)



Rys. 4.9. Projekt sie nie kompiluje

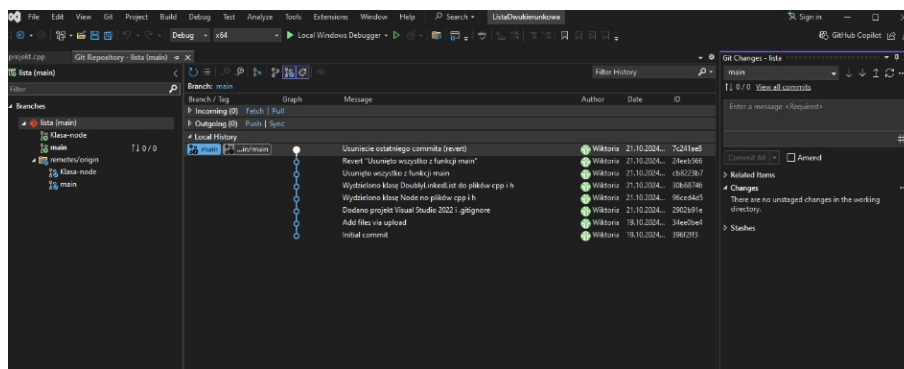
4.2.4. Dodawanie zmian w osobnych folderach

Kolejnym naszym krokiem jest reset aktualnych zmian do stanu który jest zacomitowany, aby przywrócić stan który jest w commicie. W efekcie projekt spowrotem się kompiluje. Dodajemy zmiany do main w drugim folderze. 4.10 (s. 19)



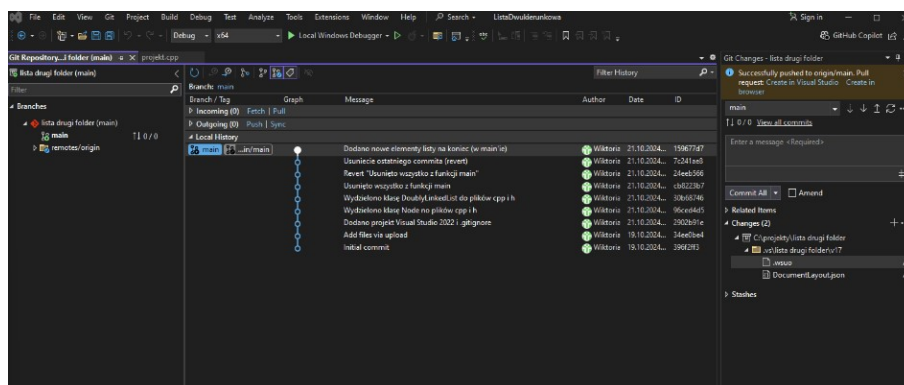
Rys. 4.10. Dodany folder

W pierwszym folderze nie widać jeszcze bieżących zmian.4.11 (s. 20).



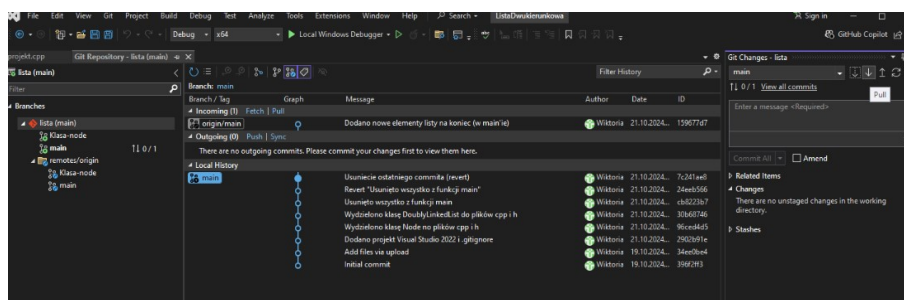
Rys. 4.11. Folder nr.1

W drugim folderze zmiany już są widoczne. 4.12 (s. 20).



Rys. 4.12. Folder nr.2

Następnie używamy pull aby pobrać nowe zmiany z serwera 4.13 (s. 20). Po wykonaniu tej czynności, projekt ma mieć taki sam stan jak drugi.



Rys. 4.13. Pobieranie zmian- pull

4.2.5. Doxyfile

Zostało nam wygenerowanie pliku doxyfile oraz folderu z plikami LaTeX 4.14 (s. 21)

```

22 cout << "Dodaj element na 1 indeks (2 miejsce): " << endl;
23 list.insertIndex(2, 1);
24 list.showList();
25
26 cout << "Wyświetlaj liste na odwrót: " << endl;
27 list.showReverseList();
28
29 cout << "Dodaj jeszcze kilka wartosci: " << endl;
30 list.insertAtEnd(30);
31 list.insertAtEnd(80);
32 list.insertAtEnd(10);
33 list.insertAtEnd(66);
34 list.insertAtEnd(97);
35 list.insertAtEnd(100);
36 list.insertAtEnd(120);
37 list.insertAtEnd(0);
38 list.showList();
39
40 cout << "Usuń element na 4 indeksie (5 miejscu): " << endl;
41 list.deleteIndex(4);
42 list.showList();
43

```

```

Program PowerShell dla developera
+ Program PowerShell dla developera
** Visual Studio 2022 Developer PowerShell v17.11.5
** Copyright (c) 2022 Microsoft Corporation
PS C:\projekty\lista\listaDwukierunkowa> doxygen

Configuration file 'Doxyfile' created.

Now edit the configuration file and enter

doxygen

to generate the documentation for your project

PS C:\projekty\lista\listaDwukierunkowa>

```

Rys. 4.14. Generowanie Doxyfile

Po wywołaniu polecenia make.bat zaczynamy generować dokumentację w PDFie. 4.15 (s. 21)

```

19 cout << "Dodaj element na koncu: " << endl;
20 list.insertAtEnd(10);
21 list.showList();
22
23 cout << "Dodaj element na 1 indeks (2 miejsce): " << endl;
24 list.insertIndex(2, 1);
25 list.showList();
26
27 cout << "Wyświetlaj liste na odwrót: " << endl;
28 list.showReverseList();
29
30 cout << "Dodaj jeszcze kilka wartosci: " << endl;
31 list.insertAtEnd(30);
32 list.insertAtEnd(80);
33 list.insertAtEnd(10);
34 list.insertAtEnd(66);
35 list.insertAtEnd(97);
36 list.insertAtEnd(100);
37 list.insertAtEnd(120);
38 list.insertAtEnd(0);
39 list.showList();
40
41 cout << "Usuń element na 4 indeksie (5 miejscu): " << endl;
42 list.deleteIndex(4);
43 list.showList();
44

```

```

Program PowerShell dla developera
+ Program PowerShell dla developera
PS C:\projekty\lista\listaDwukierunkowa\latex> ls

Directory: C:\projekty\lista\listaDwukierunkowa\latex

Mode                LastWriteTime         Length Name
----                -
-a----- 21.10.2024 20:47           374 annotated.tex
-a----- 21.10.2024 20:47          2141 class doubly_linked_list.tex
-a----- 21.10.2024 20:47          1146 class_node.tex
-a----- 21.10.2024 20:47          2154 doxygen.sty
-a----- 21.10.2024 20:47          75581 etox_doxygen.sty
-a----- 21.10.2024 20:47           387 files.tex
-a----- 21.10.2024 20:47          12723 longtable_doxygen.sty
-a----- 21.10.2024 20:47           1919 make.bat
-a----- 21.10.2024 20:47          1336 Makefile
-a----- 21.10.2024 20:47           7493 refman.tex
-a----- 21.10.2024 20:47          114906 tabu_doxygen.sty
-a----- 21.10.2024 20:47          2217 doubly_linked_list_8h_source.tex
-a----- 21.10.2024 20:47           803 _node_8h_source.tex

PS C:\projekty\lista\listaDwukierunkowa\latex>

```

Rys. 4.15. Dokumentacja PDF

5. Wnioski

W ramach projektu zaimplementowano listę dwukierunkową w języku C++, która umożliwia dodawanie, usuwanie oraz przeglądanie elementów zarówno od początku, jak i końca listy. Kluczowe operacje, takie jak wstawianie na dowolnej pozycji, usuwanie i czyszczenie listy, zostały poprawnie zrealizowane z uwzględnieniem zarządzania dynamiczną pamięcią, co zapobiega wyciekom. W projekcie wykorzystano system kontroli wersji Git oraz platformę GitHub, co pozwoliło na efektywne zarządzanie wersjami kodu i łatwe śledzenie zmian. Doxygen posłużył do automatycznego generowania dokumentacji technicznej na podstawie kodu, co ułatwiło organizację i przegląd projektu. Dzięki integracji Gita, GitHuba i Doxygena projekt był dobrze zorganizowany i umożliwiał wygodną współpracę oraz utrzymanie porządku w dokumentacji.

Bibliografia

- [1] *Strona internetowa GEEKSFORGEEEKS*. URL: <https://www.geeksforgeeks.org/doubly-linked-list/> (term. wiz. 22.10.2024).
- [2] *Strona internetowa Samouczek Programisty*. URL: <https://www.samouczekprogramisty.pl/struktury-danych-lista-wiazana/> (term. wiz. 23.10.2022).
- [3] *Strona internetowa Deviniti*. URL: <https://deviniti.com/pl/blog/devops-pl/jak-korzystac-z-git/> (term. wiz. 22.10.2024).

Spis rysunków

2.1. Reprezentacja listy dwukierunkowej.	6
2.2. Wstawianie na początku listy dwukierunkowo powiązanej.	7
3.1. UML	9
4.1. Cofnięcie 1.1	14
4.2. Cofnięcie 1.2	15
4.3. Branch 1.1	15
4.4. Branch 1.2	16
4.5. Wizualizacja branchy	16
4.6. Commit-usuwany	17
4.7. Revert commita	17
4.8. Projekt	18
4.9. Projekt sie nie kompiluje	18
4.10. Dodany folder	19
4.11. Folder nr.1	20
4.12. Folder nr.2	20
4.13. Pobieranie zmian- pull	20
4.14. Generowanie Doxyfile	21
4.15. Dokumentacja PDF	21

Spis tabel

Spis listingów

1.	Klasa Node	10
2.	Dodawanie elementów na początku listy	11
3.	Dodawanie elementów na końcu listy	11
4.	Usuwanie elementu	12
5.	lista	13
6.	Wynik	13